

JS I DOM NA PRZYKŁADZIE LISTY TODO

SPIS TREŚCI

Spis treści.....	1
Cel zajęć.....	1
Rozpoczęcie.....	1
Uwaga.....	2
Wymagania.....	2
Strona HTML.....	3
Klasa Todo.....	3
Dodawanie pozycji listy.....	3
Usuwanie pozycji listy.....	4
Edycja pozycji listy.....	4
Odczyt / Zapis LocalStorage.....	5
Wyszukiwanie.....	6
Commit projektu do GIT.....	6
Podsumowanie.....	7

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- przemieszczania się po drzewie DOM;
- dodawania, usuwania, edytowania elementów drzewa DOM.

W praktycznym wymiarze utworzona zostanie dynamiczna lista czynności do zrobienia (lista To Do).

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie metod przemieszczania się po drzewie DOM.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

WYMAGANIA

W ramach LAB B przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS
- lista zadań
- na dole listy pole tekstowe do dodawania nowych zadań, pole typu data/czas do określenia terminu wykonania zadania, przycisk dodawania zadania
- walidacja nowych zadań: co najmniej 3 znaki, nie więcej niż 255 znaków, data musi być pusta albo w przyszłości
- na górze listy pole wyszukiwarki
- po wpisaniu w wyszukiwarkę co najmniej 2 znaków na liście wyświetlają się wyłącznie pozycje zawierające wpisaną w wyszukiwarkę frazę
- wyszukiwana fraza zostaje wyróżniona w każdym wyniku wyszukiwania
- kliknięcie na dowolną pozycję listy zmienia ją w pole edycji; kliknięcie poza pozycję listy zapisuje zmiany
- obok każdej pozycji listy znajduje się przycisk Usun / Śmietnik
- wpisy na liście zapisują się do Local Storage
- po odświeżeniu strony lista wypełnia się wpisami z Local Storage

Mockupy:

Mockup of the application interface showing a list of tasks. The interface includes a search bar at the top. Below it, a list of tasks is displayed, each with a checkbox, a name, a date, and a delete button (trash icon). The tasks are: chocolate (2000-01-01), macaroon, chupa chups, candy canes (2000-01-05), and bon bons. At the bottom, there is a text input field labeled 'do zrobienia...', a date input field with a calendar icon and the date '2000-01-01', and a 'Zapisz' button.

Mockup of the application interface showing a task being edited. The interface is similar to the previous one, but the 'chupa chups' task is selected, and its name is highlighted in a text input field. A 'Zapisz' button is visible next to the input field. The date input field at the bottom shows '2000-01-01'.

Mockup of the application interface showing search results. The search bar contains the text 'on'. The list of tasks is filtered to show only those containing 'on': macaroon and bon bons. The date input field at the bottom shows '2000-01-01'.

STRONA HTML

Prace rozpocznij od implementacji HTML z danymi wpisanymi „na sztywno”. Upewnij się, że wstawione zostały wszystkie wymagane elementy – pole wyszukiwarki, lista, pole dodawania, przycisk usuwania. To laboratorium koncentruje się na JS, więc może być ładne, ale nie musi. **Nie trać za dużo czasu na CSS** – to jest laboratorium z JS.

Wstaw zrzut ekranu przedstawiający stronę HTML z polem wyszukiwarki, listą, polem dodawania, przyciskami usuwania:

The screenshot shows a web form with a search bar labeled "Szukaj..." with a magnifying glass icon. Below it is a list item with the text "placeholder" and "2025-10-24", and a close button with an "X" icon. Below the list item are two input fields: "placeholder-2" and "dd . mm . rrrr" with a calendar icon. At the bottom are three buttons: "do zrobienia...", "dd . mm . rrrr" with a calendar icon, and "Zapisz :)".

Punkty:	0	1
---------	---	---

KLASA TODO

Pierwszym instynktem może być chęć dodania zachowań bezpośrednio do elementów listy w drzewie DOM. Chociaż na krótką metę wydaje się być to najprostsze rozwiązanie, za chwilę okaże się krótkowzroczne i trudne do implementacji przy kolejnych punktach 😊

Najlepszym sposobem rozwiązania tego laboratorium jest utworzenie klasy Todo (albo po prostu obiektu z kilkoma metodami). Bez względu na przyjętą strategię, należy w tym nowoutworzonym bycie utworzyć tablicę `tasks` oraz metodę `draw()`, która wyczyści `div` z obecną wizualizacją zadań do zrobienia i wygeneruje ją na nowo na podstawie tablicy `tasks`.

W celu sprawdzenia poprawności działania, najlepiej dostać się do tablicy `tasks` i edytować jej zawartość, po czym ręcznie wywołać metodę `draw()`. Jeśli zawartość listy wyrenderuje się na nowo poprawnie – możemy iść dalej!

Zaimplementuj dodawanie, usuwanie, edycję pozycji listy – wszystko modyfikujące tablicę `tasks` i wywołujące na koniec metodę `draw()`.

DODAWANIE POZYCJI LISTY

Wstaw zrzut ekranu listy przed dodaniem nowego zadania:

The screenshot shows the top part of the web form, including the search bar "Szukaj..." and the "do zrobienia..." button. The list item and the input fields are not visible in this view.

Wstaw zrzut ekranu listy po dodaniu nowego zadania:

 Szukaj...

Idź pobięgać

2025-10-28

X

do zrobienia...

dd . mm . rrrr 

Zapisz :)

Punkty:	0	1
---------	---	---

USUWANIE POZYCJI LISTY

Wstaw zrzut ekranu listy przed usunięciem wybranego zadania:

 Szukaj...

Idź pobięgać

2025-10-28

X

fgdgfsdgsdgsdf

2025-10-31

X

wysprzątać szope

2025-10-27

X

do zrobienia...

dd . mm . rrrr 

Zapisz :)

Wstaw zrzut ekranu listy po usunięciu zadania:

 Szukaj...

Idź pobięgać

2025-10-28

X

wysprzątać szope

2025-10-27

X

do zrobienia...

dd . mm . rrrr 

Zapisz :)

Punkty:	0	1
---------	---	---

EDYCJA POZYCJI LISTY

Wstaw zrzut ekranu listy przed edycją wybranego zadania:

 Szukaj...

Idź pobiegać

2025-10-28

X

wypadsada szope

2025-10-27

X

do zrobienia...

dd . mm . rrrr 

Zapisz :)

Wstaw zrzut ekranu listy w trakcie edytowania zadania i daty:

 Szukaj...

Idź pobiegać

2025-10-28

X

wysprzą szope

27 . 10 . 2025 

X

do zrobienia...

dd . mm . rrrr 

Zapisz :)

Wstaw zrzut ekranu listy po edycji zadania i daty. Upewnij się, że dane się zapisały i zadanie jest zmienione:

 Szukaj...

Idź pobiegać

2025-10-28

X

wysprzątać szope

2025-10-26

X

do zrobienia...

dd . mm . rrrr 

Zapisz :)

Punkty:

0

1

ODCZYT / ZAPIS LOCALSTORAGE

Zastosowanie klasy Todo w realizacji tego laboratorium pozwala w bardzo łatwy sposób odczytywać i zapisywać stan listy do pamięci przeglądarki. Wystarczy serializacja / deserializacja za pomocą JSON.parse() i JSON.stringify().

Wstaw rzuty ekranu przedstawiające wygląd listy i zawartość local storage gdy na liście są pewne zadania:

Lokalna pamięć

http://localhost:63342

Pamięć podręczna

Pamięć sesji

Szukaj...

Pobiegac
2025-10-25

Sprzątnąć szope
2025-10-27

do zrobienia... dd . mm . rrrr Zapisz :)

Key	Value
local_storage	[{"text":"Pobiegac","date":"2025-10-25"}, {"text":"Sprzątnąć szope","date":"2025-10-27"}]
storageText	

Filtruj wartości

Dane

local_storage:[{"text":"Pobiegac","date":"2025-10-25"}, {"text":"Sprzątnąć szope","date":"2025-10-27"}]

Przetworzona wartość

local_storage:Array

0:Object

text:"Pobiegac"

date:"2025-10-25"

__proto__:Object

1:Object

text:"Sprzątnąć szope"

date:"2025-10-27"

__proto__:Object

length:2

__proto__:Array

Wstaw rzuty ekranu przedstawiające wygląd listy i zawartość local storage po dodaniu nowej pozycji listy. Upewnij się, że widoczne w local storage są dane dotyczące nowego zadania:

Lokalna pamięć

http://localhost:63342

Pamięć podręczna

Pamięć sesji

Szukaj...

Pobiegac
2025-10-25

Sprzątnąć szope
2025-10-27

Cos mam zrobić, nie wiem co
2025-10-31

do zrobienia... dd . mm . rrrr Zapisz :)

Key	Value
local_stor...	[{"text":"Pobiegac","date":"2025-10-25"}, {"text":"Sprzątnąć szope","date":"2025-10-27"}, {"text":"Cos mam zrobić, nie wiem co","date":"2025-10-31"}]
storageText	

Filtruj wartości

Dane

local_storage:[{"text":"Pobiegac","date":"2025-10-25"}, {"text":"Sprzątnąć szope","date":"2025-10-27"}, {"text":"Cos mam zrobić, nie wiem co","date":"2025-10-31"}]

Przetworzona wartość

local_storage:Array

0:Object

text:"Pobiegac"

date:"2025-10-25"

__proto__:Object

1:Object

text:"Sprzątnąć szope"

date:"2025-10-27"

__proto__:Object

2:Object

text:"Cos mam zrobić, nie wiem co"

date:"2025-10-31"

__proto__:Object

length:3

__proto__:Array

Punkty:	0	1
---------	---	---

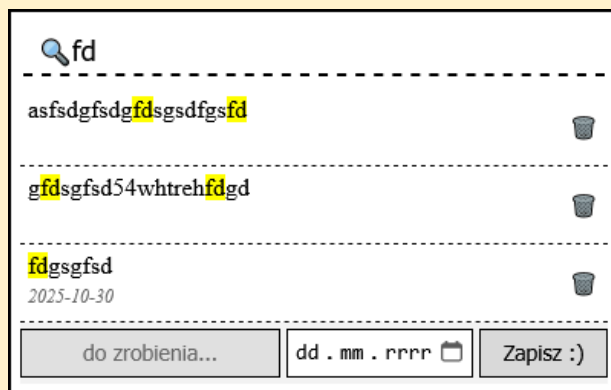
WYSZUKIWANIE

Na koniec zostało filtrowanie wyników. Proponowanym podejściem do tego tematu jest umieszczenie w klasie `Todo` właściwości `term` – frazy wyszukiwanej przez użytkownika. Następnie można utworzyć metodę `getFilteredTasks`, albo getter `filteredTasks`, która zwracać będzie te elementy tablicy `tasks`, które odpowiadają zapytaniu. Można użyć funkcji wyższego rzędu `filter()`.

Wstaw zrzut ekranu listy, gdy pole wyszukiwania jest puste:



Wstaw zrzut ekranu listy, gdy w polu wyszukiwania wpisano wystarczająco dużo znaków, by zadziałało filtrowanie. Upewnij się, że chociaż 2 wyniki będą wciąż widoczne:

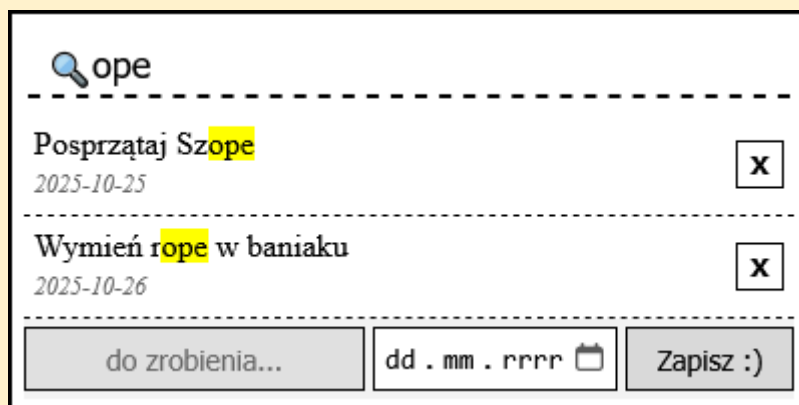


Punkty:

0

1

Wstaw zrzut ekranu przedstawiający podświetlenie szukanej frazy w wynikach wyszukiwania, przykładowo dla frazy ko i zadania Ala ma kota otrzymujemy: Ala ma **kota**:



Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie **lab-b** na podstawie głównej gałęzi kodu.

Podaj link do brancha **lab-b** w swoim repozytorium:

...link, np. <https://github.com/inazwisko/ai1-lab/tree/lab-b...>

<https://github.com/jaqbqlik/5-semestr-Aplikacje-Internetowe-/tree/main/lab-b>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

Ogólnie ciekawe dość laboratorium...

Ale zadanie

„Wstaw zrzut ekranu listy, gdy w polu wyszukiwania wpisano wystarczająco dużo znaków, by zadziałało filtrowanie.

Upewnij się, że chociaż 2 wyniki będą wciąż widoczne: „

oraz

„Wstaw zrzut ekranu przedstawiający podświetlenie szukanej frazy w wynikach wyszukiwania, przykładowo dla frazy ko i zadania Ala ma kota otrzymujemy: Ala ma kota:”

Wydają się dosyć podobne do siebie, no chyba że coś źle zrozumiałem.

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.